



Dennis Janka

Sequenzielle Entscheidungsprobleme

Sequenzielle Entscheidungsprobleme: Beispiele

Mathematische Modellierung von sequenziellen Entscheidungsproblemen

Lösungsansätze / Policies

Siehe neues Kapitel im Skript:

<https://193-196-37-212.sslip.io/ml/sequenzielle-entscheidungsprobleme/>

Entscheidungsprobleme: Beispiele



Beispiel 1: Brettspiele



Entscheidungen

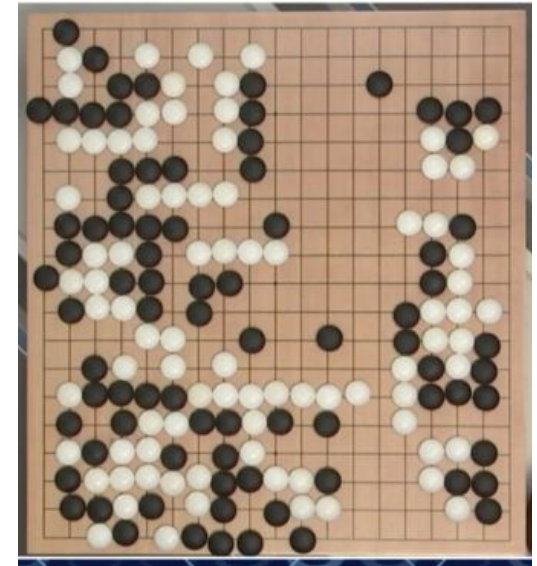
- + Welchen Zug sollte man in einer gegebenen Situation machen?

Ziel

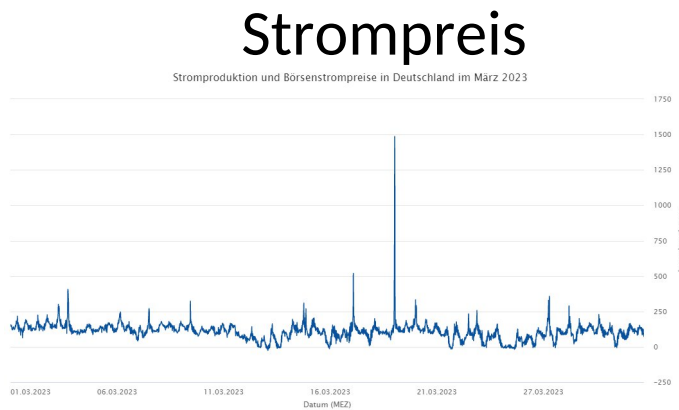
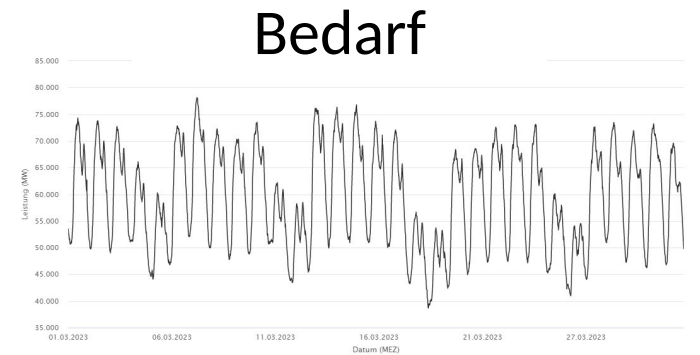
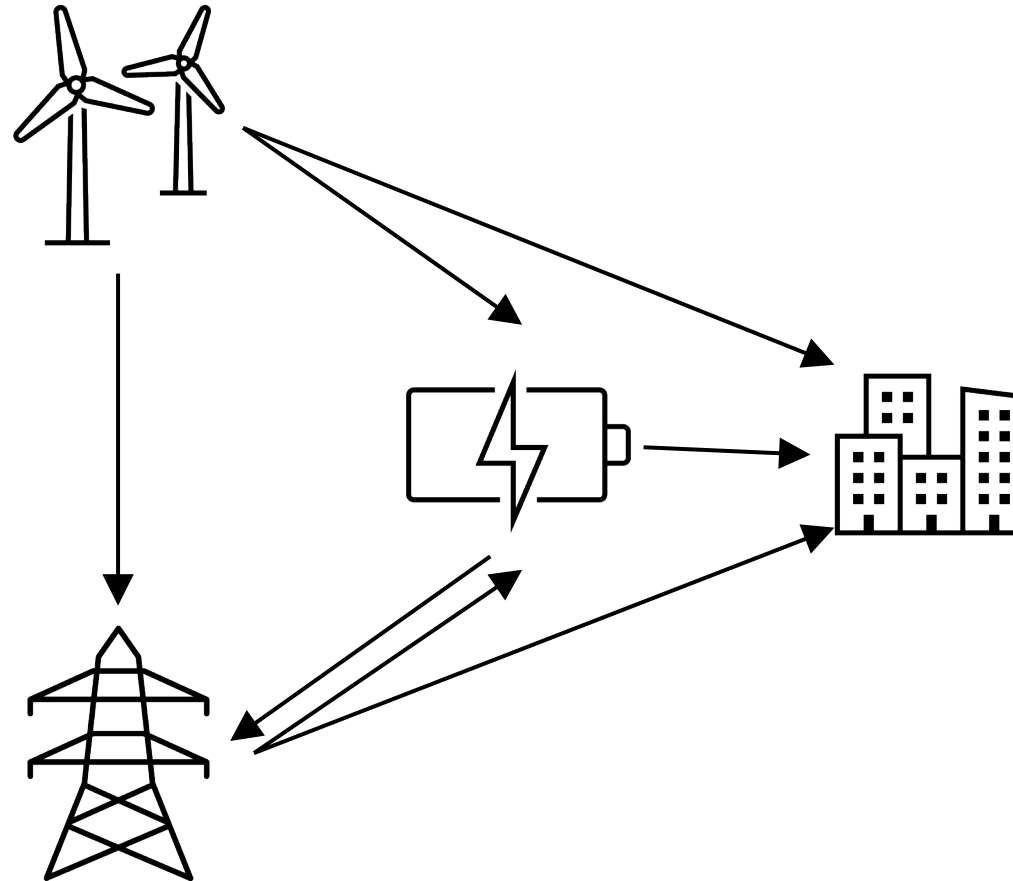
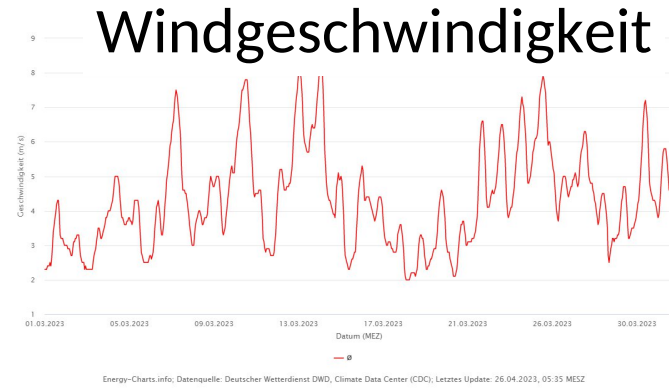
- + Das Spiel gewinnen

Unsicherheiten

- + Was macht der Gegner als nächstes?



Beispiel 2: Energiemanagement



Beispiel 2: Energiemanagement



Entscheidungen

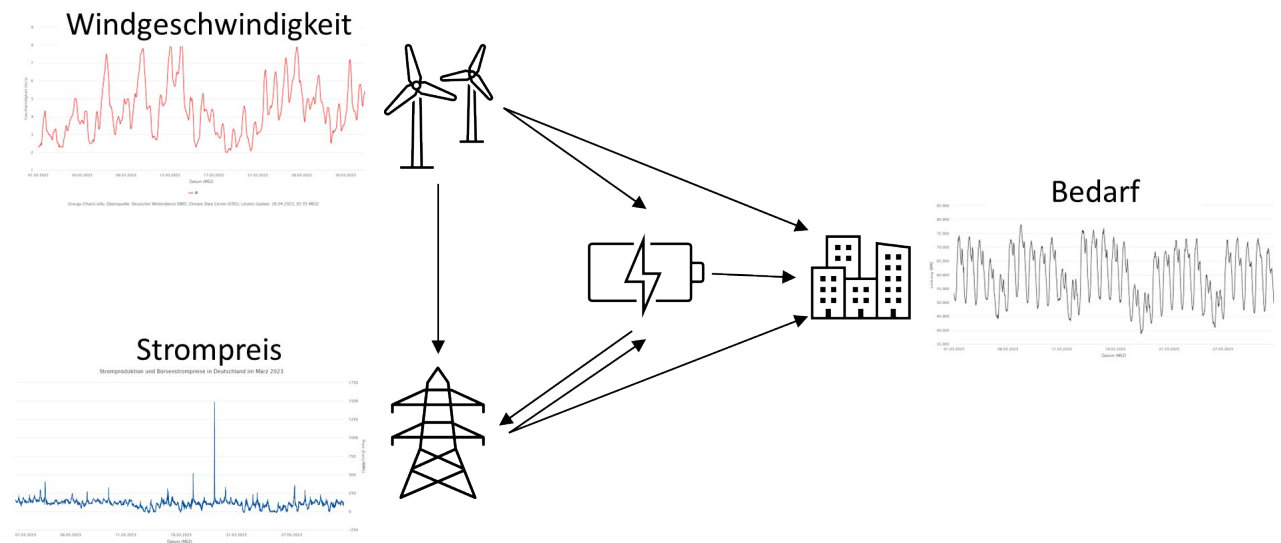
- + Wie viel Strom soll zum Zeitpunkt t aus der Anlage/der Batterie/dem Netz bezogen werden?
- + Wie viel Strom aus der Anlage/dem Netz soll zum Zeitpunkt t in die Batterie eingespeist werden?
- + Wie viel Strom aus der Anlage/der Batterie soll zum Zeitpunkt t in das Netz eingespeist werden?

Ziel

- + Zu jeder Zeit möglichst kostengünstig den Bedarf decken

Unsicherheiten

- + Wetter
- + Strompreis
- + Strombedarf



Beispiel 3: Navigationssysteme



Entscheidungen

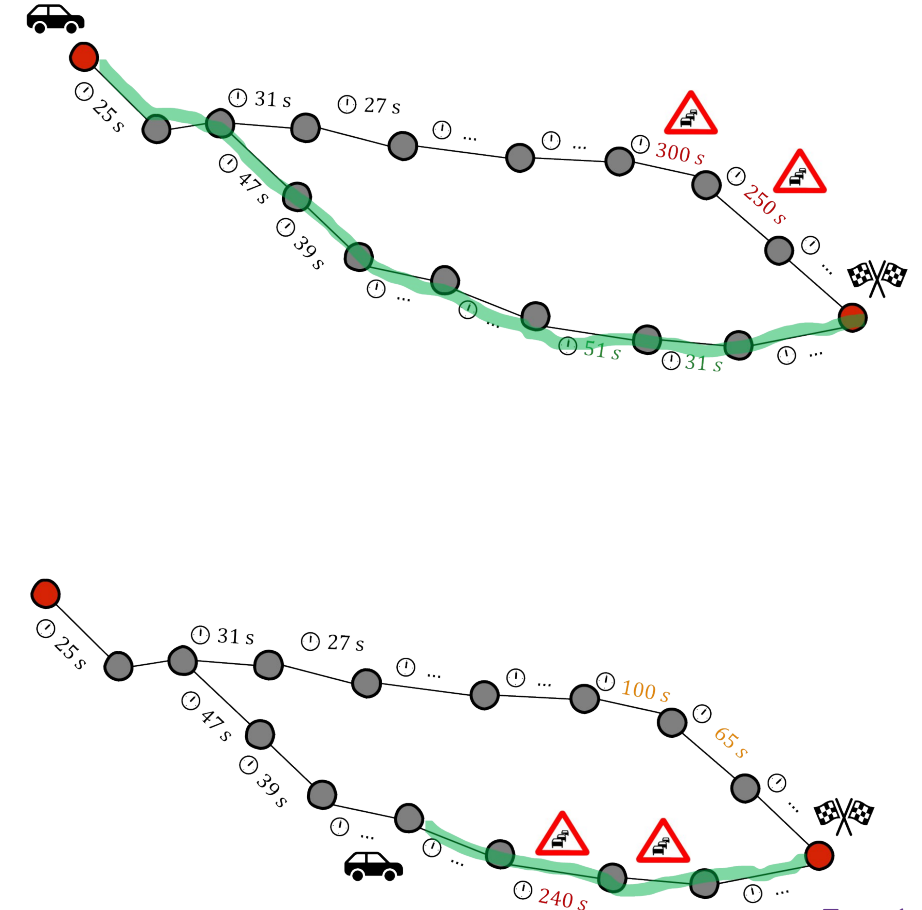
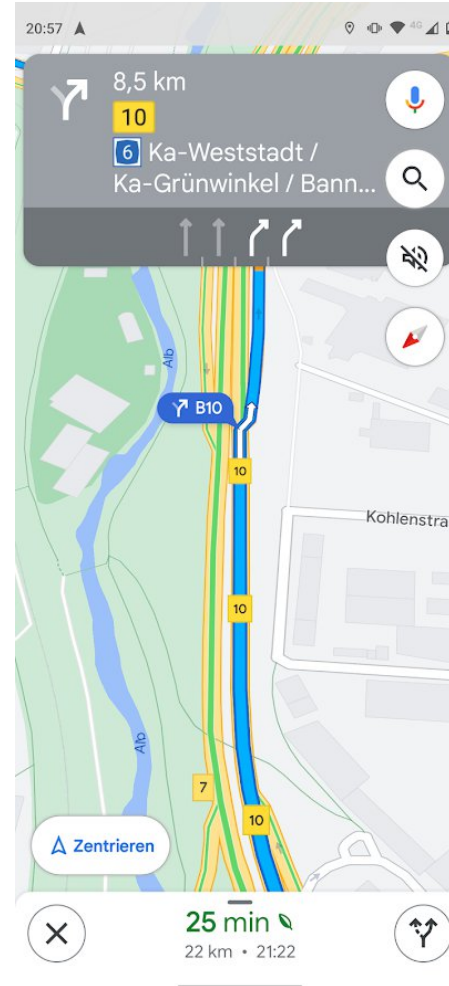
+ Welche Route soll weiter verfolgt werden?

Ziel

+ Möglichst schnell ans Ziel kommen

Unsicherheiten

+ Verkehrsbedingungen



K
A

Beispiel 4: Lademanagement im Bergbau



Hintergrund

Maschinen im Bergbau benötigen sehr viel Treibstoff

Abgase unter Tage müssen durch aufwendige Belüftung nach außen geführt werden

In vielen Gegenden, in denen Bergbau betrieben wird, gibt es großes Potential für erneuerbare Energien



500 L Diesel / h!!



Herausforderung (u.a.): Management von Ladeoperationen

- + Welches Fahrzeug?
- + Wann?
- + Wo?
- + Wie lange?

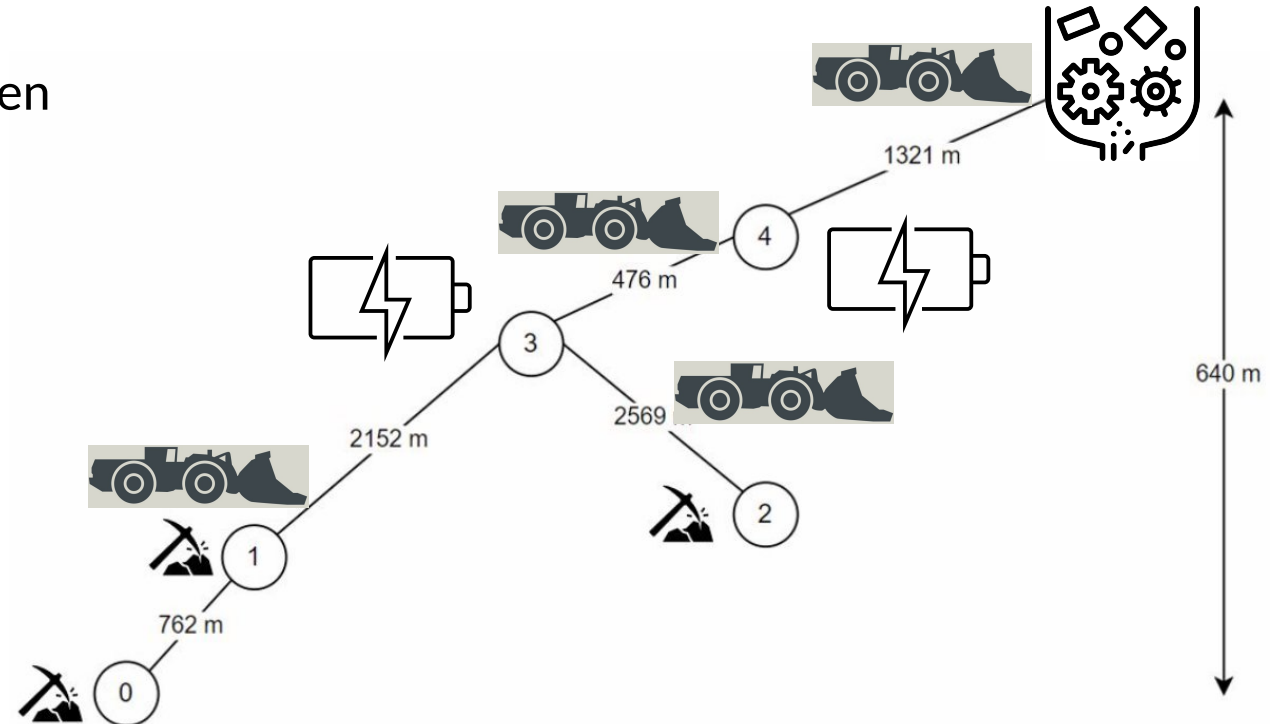


Beispiel 4: Lademanagement im Bergbau



Szenario

- + Fahrzeuge sollen Material von Steinbrüchen zum Crusher bringen
- + Batterien müssen zwischendurch aufgeladen werden
- + Ladestationen an verschiedenen Stellen
- + Wartezeiten vermeiden
- + Stillstand/leere Batterie vermeiden
- + Kapazität des Stromnetzes beachten



Beispiel 4: Lademanagement im Bergbau



Entscheidungen

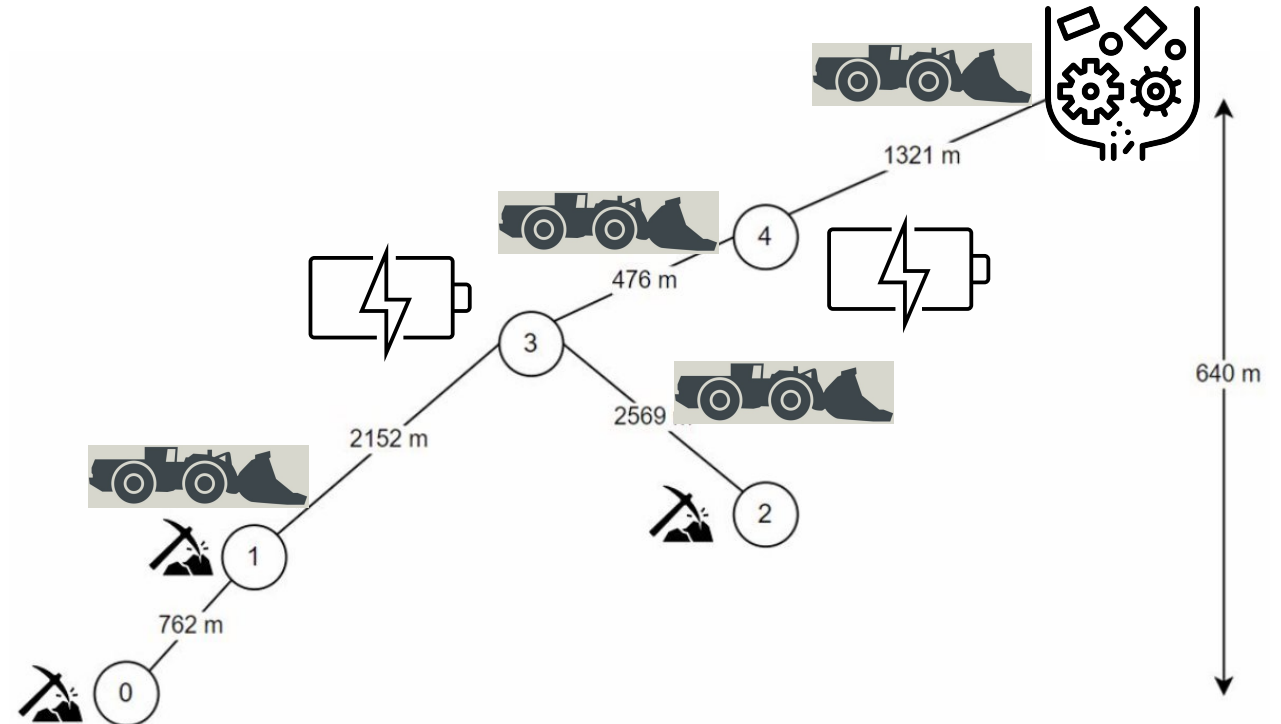
- + Was soll Truck n als nächstes machen? (Batterie an Ladesäule m laden / Material aufladen / Material abladen)

Ziel

- + Möglichst viel Material fördern oder vorgegebene Menge mit möglichst geringem Aufwand fördern

Unsicherheiten

- + Geschwindigkeit der Fahrzeuge
- + Ladestand der Batterie



Was haben alle Beispiele gemeinsam?



- + Alle Probleme haben die Form

Entscheidung, Information, Entscheidung, Information, ...

- + Performancemetrik (Ziel) für (langfristige) Bewertung von Entscheidungen
- + Unsicherheiten
- + Entscheidungen müssen situationsabhängig getroffen werden

Weitere Beispiele

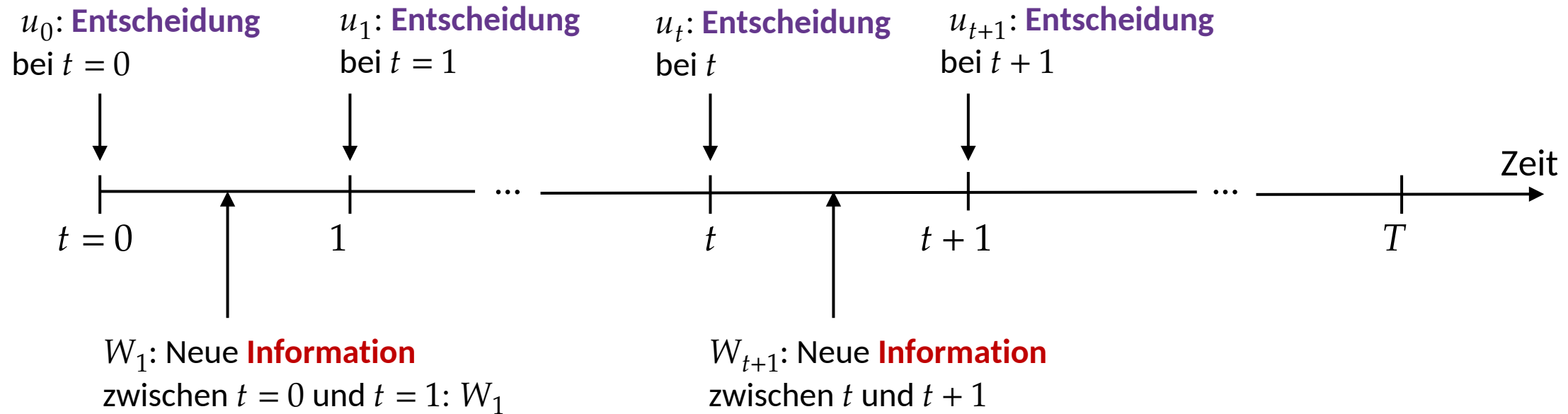


Gebiet	Entscheidungen
Produktionsplanung	Wie viel Essen soll die Mensa zubereiten, so dass alle Kunden versorgt werden aber am Ende möglichst wenig übrigbleibt?
Gesundheitswesen	Wie sollen Tests durchgeführt werden, um den Verlauf einer Pandemie einzuschätzen? Welche Bevölkerungsgruppen sollen vorrangig geimpft werden?
Energie	Wann müssen welche Erzeuger und Verbraucher zu- bzw. abgeschaltet werden, damit das Stromnetz stabil ist?
VWL / Politik	Wie sollte die EZB den Leitzins festlegen, gegeben die aktuelle wirtschaftliche Situation? Welche Produkte/Dienstleistungen sollten wie subventioniert werden?
Business	Welchen Preis soll man für ein Produkt verlangen?
Marketing	Welche Werbung sollte man anzeigen, um die Klickrate zu maximieren?
...	...

Mathematische Modellierung



Entscheidung, Information, Entscheidung, Information, ...



Intervalle: z.B. Tage, Stunden, Sekunden, Spielzügen, Schritte

Index t : Größe ist zum Zeitpunkt t bekannt

Ein **sequenzielles Entscheidungsproblem** besteht aus **fünf** Komponenten:

1. *Zustandsvariablen* x_t :
Alles, was wir zum Zeitpunkt t wissen müssen, um eine Entscheidung zu treffen
2. *Entscheidungsvariablen* u_t ,
die durch Policies $u_t = \pi(x_t)$ getroffen werden. Wie Policies funktionieren —> später!
3. *Exogene Information* W_{t+1} :
Alles, was wir zwischen t und $t + 1$ neu erfahren
4. *Zustandsübergangsfunktion* $x_{t+1} = F(x_t, u_t, W_{t+1})$:
Gleichungen, die beschreiben, wie sich das System über die Zeit entwickelt (Modellfunktion)
5. *Rewardfunktion*: Jede Entscheidung lässt sich unmittelbar durch eine Zahl $C(x_t, u_t, W_{t+1})$ bewerten.
Ziel ist es, die *erwartete* Summe aller Beiträge zu maximieren:

$$\max_{u_t} \mathbb{E} \left(\sum_{t=0}^T C(x_t, u_t, W_{t+1}) \right)$$

Beispiel: Energiemanagement

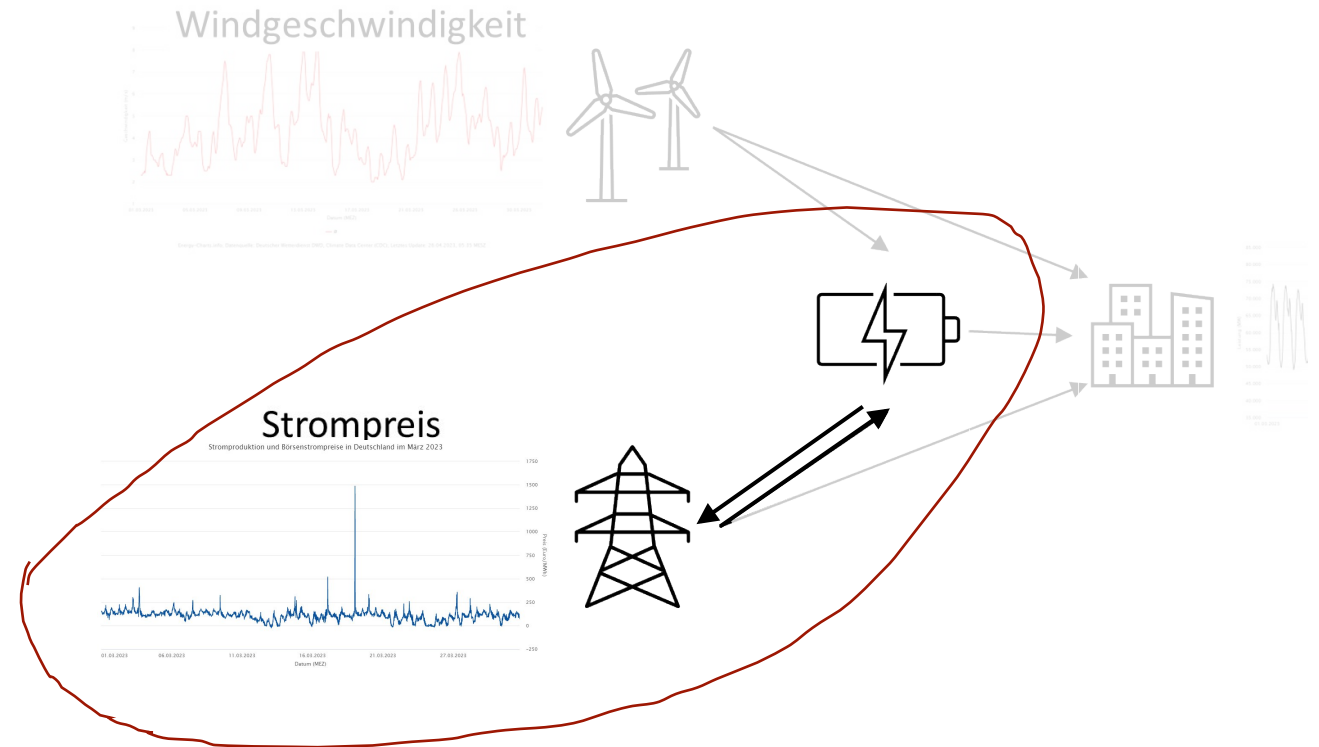


Vereinfachtes System: „Batterie-Arbitrage“

Zu jedem Zeitpunkt t kann Strom zum aktuellen Marktpreis ein- oder gespeichert werden

Strompreis sehr variabel, richtet sich nach Angebot und Nachfrage

Ziel: Möglichst viel Gewinn durch Kauf und Verkauf von Strom



1. Zustandsvariablen $x_t = (S_t, p_t)$

- S_t : Ladestand der Batterie
- p_t : aktueller Strompreis

2. Entscheidungsvariablen u_t

- Strom wird gekauft/Batterie geladen ($u_t > 0$)
bzw. verkauft/Batterie entladen ($u_t < 0$)

- Bedingungen:

$$u_t \leq S^{max} - S_t$$

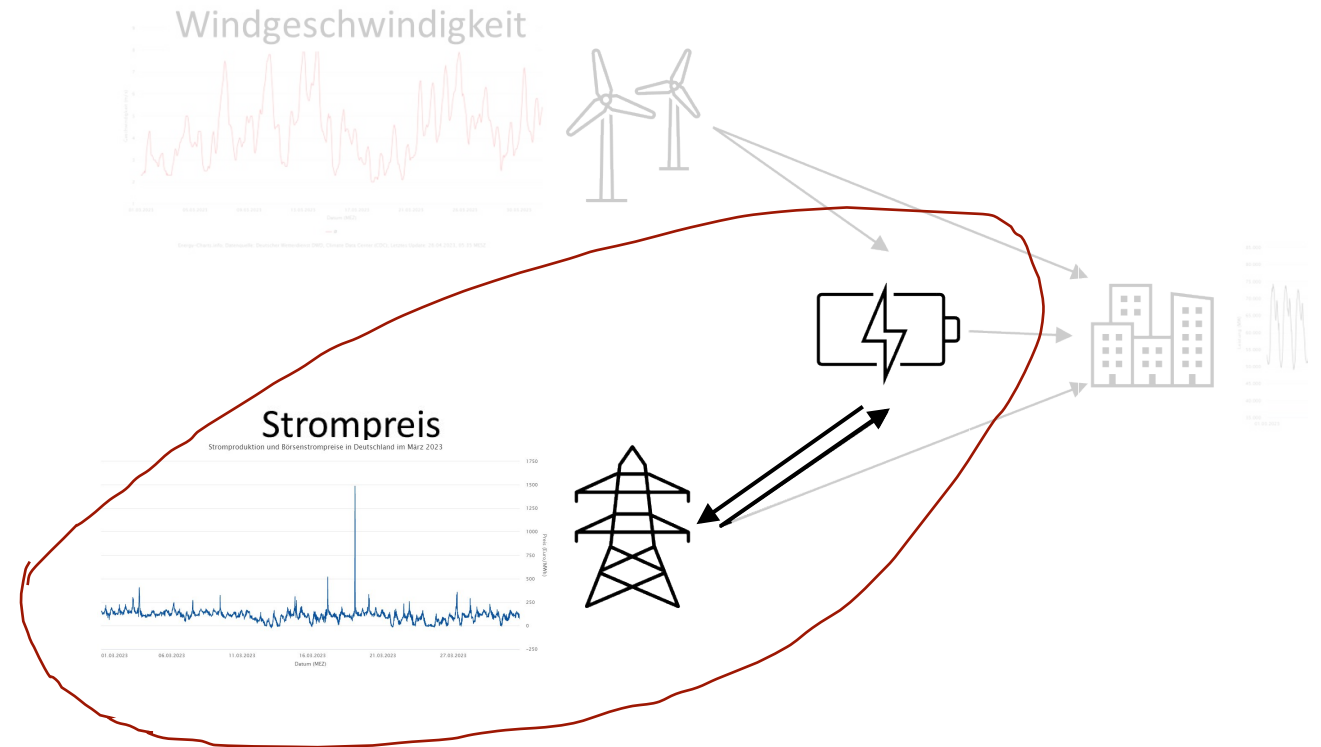
(Batterie kann nicht beliebig viel speichern)

$$u_t \geq -S_t$$

(Eine leere Batterie kann man nicht entladen)

$$u_{min} \leq u_t \leq u_{max}$$

(Man kann nicht beliebig schnell laden/entladen)



3. Exogene Information:

- Strompreis p_t

4. Zustandsübergangsfunktion:

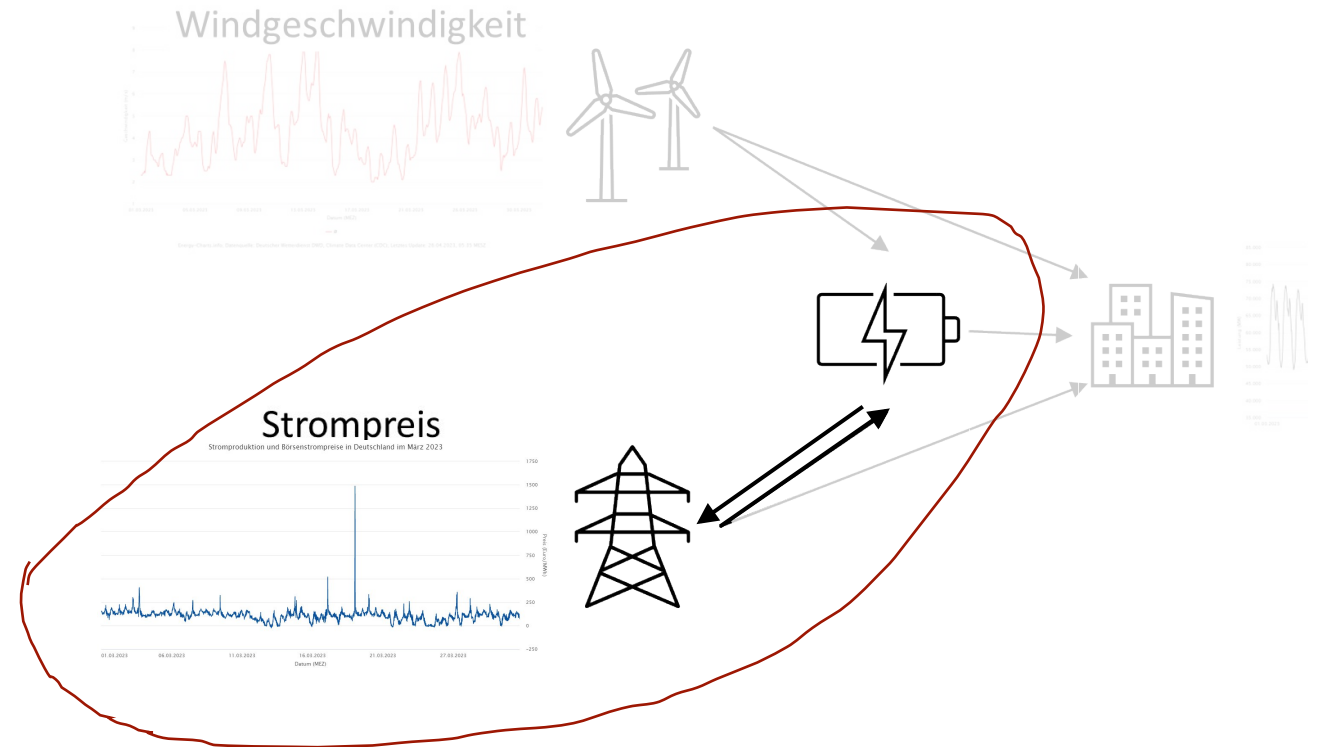
- $S_{t+1} = S_t + u_t$

5. Rewardfunktion:

- Gewinn maximieren:

$$\max_{u_t} \mathbb{E} \sum_{t=0}^T \underbrace{-p_t u_t}_{\text{Reward}}$$

Policy: Wie viel ein-/ausspeichern,
abhängig von Ladestand und Preis



Policies





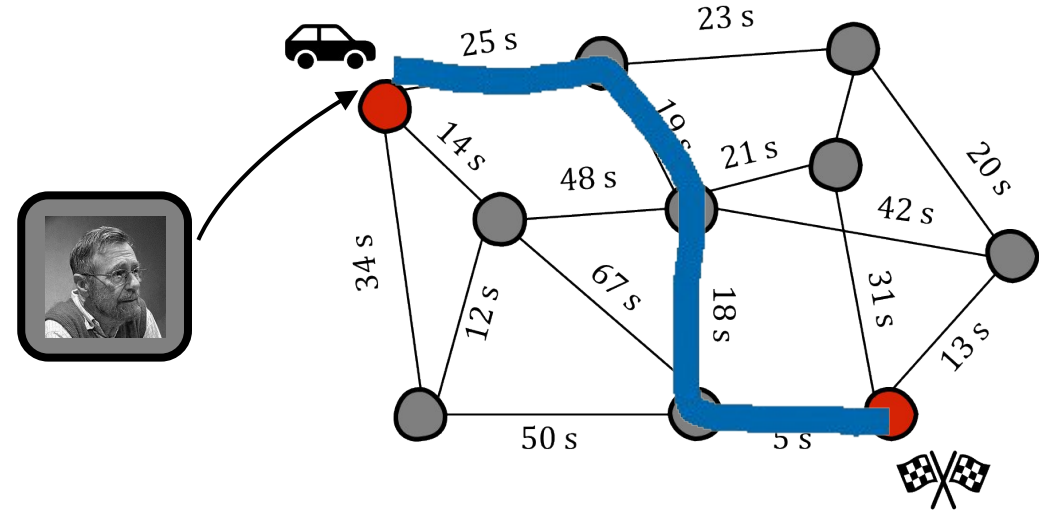
1. Parametrische Approximation
2. Model-predictive Control (MPC)
3. Approximation der Value Funktion („Reinforcement Learning“)

Beispiel: Navigationssysteme

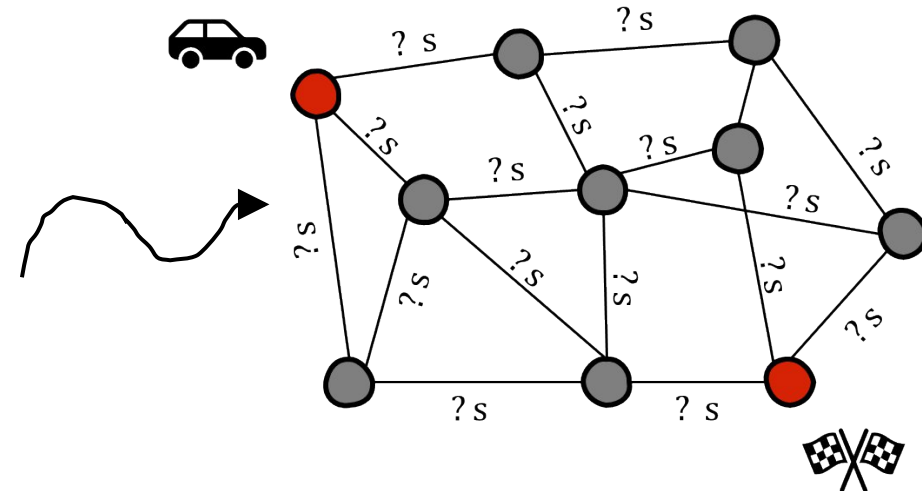


Schnellster Weg von A nach B ist ein Problem auf **Graphen**, sog. *Kürzeste-Wege-Problem*

Kann effizient gelöst werden, z.B. mit Algorithmus von Dijkstra (1959)



Situation Navigationssystem: **Kantengewichte unbekannt** und **verändern** sich ständig!



KA

Beispiel: Navigationssysteme



1. Zustandsvariablen x_t :

- Aktuelle Position
- Vorhersage Kantengewicht aller Kanten für verschiedene Zeithorizonte

2. Entscheidungsvariablen u_t ,

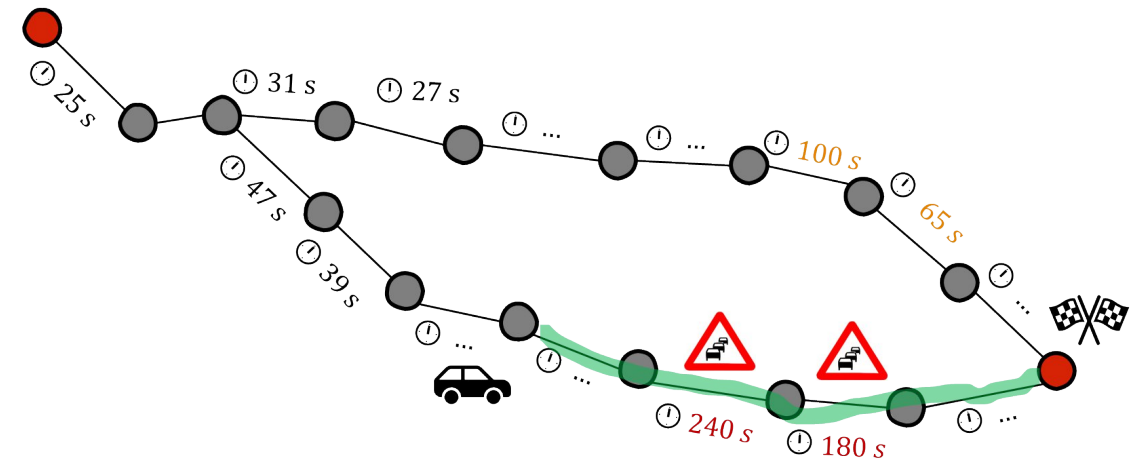
- Soll ich Kante ij durchqueren (0/1)?
- Bedingung: Kante ist an der aktuellen Position

3. Exogene Information W_{t+1} :

- Wie lange habe ich für die gerade durchquerte Kante gebraucht?
- Update Vorhersage Kantengewicht aller Kanten

4. Zielfunktion: Minimiere den Erwartungswert der Summe der Kantengewichte, die auf dem Weg zum Ziel durchquert wurden.

← Riesiger Zustandsraum!



Exogene Information W_{t+1}
kommt aus ML Modell

Policy Navigationssysteme

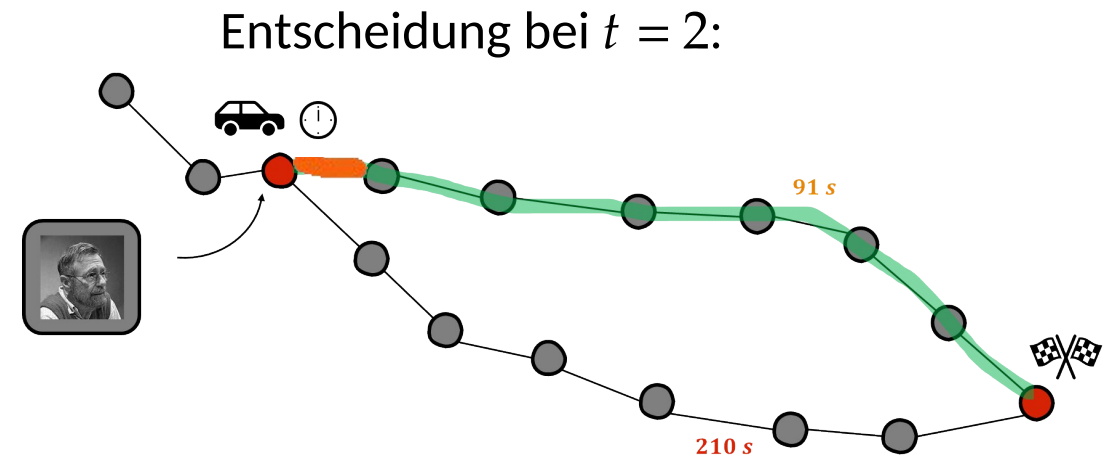
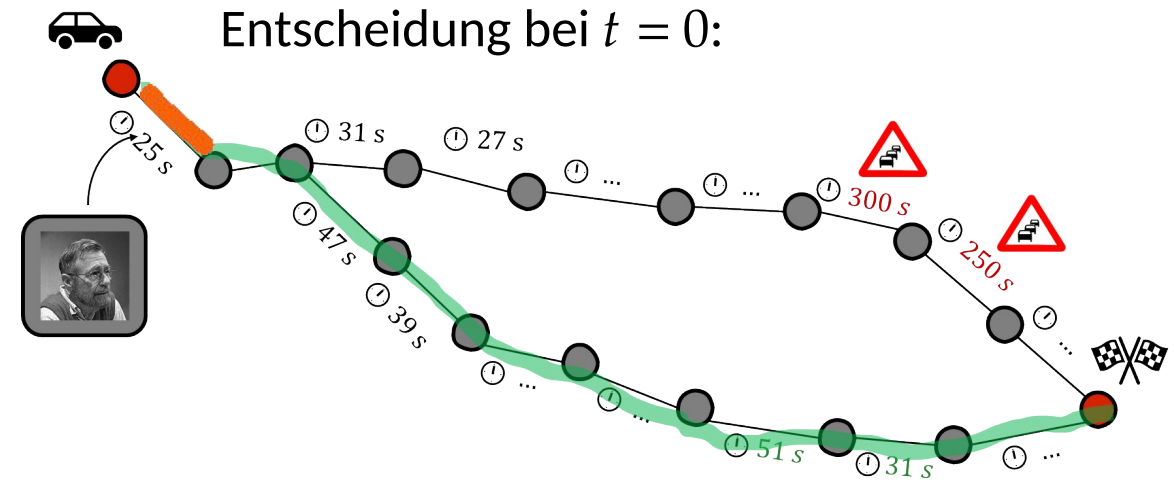


Policy, die in den meisten Systemen verfolgt wird:

Zu jedem Zeitpunkt (immer, wenn eine neue Vorhersage für das Kantengewicht verfügbar ist)

1. Tue so, als wäre die neue Vorhersage korrekt
2. Löse das deterministische kürzeste-Wege-Problem
3. Fahre so lange auf dem kürzesten Weg aus 2., bis eine neue Vorhersage verfügbar ist. Gehe dann zu 1.

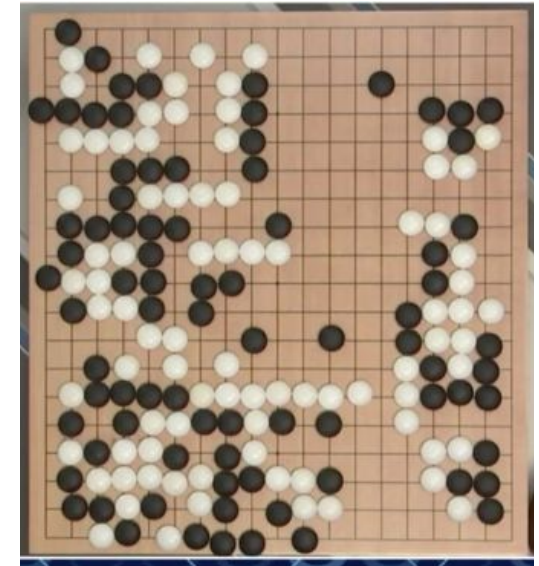
—> Beim Anwenden der Policy muss jedes Mal ein Optimierungsproblem gelöst werden.



Beispiel: Brettspiele



1. *Zustandsvariablen x_t* :
 - Belegung der einzelnen Felder
 - Evtl. Belegung der Felder in den vorangegangenen Zügen
2. *Entscheidungsvariablen u_t* ,
 - Welcher Zug soll gemacht werden?
 - Bedingung: Zug muss zulässig sein
3. *Exogene Information W_{t+1}* :
 - Welchen Zug hat der Gegner als letztes gemacht?
4. *Zielfunktion*: Beitrag nach jedem Zug:
 - + 1, wenn das Spiel vorbei ist und gewonnen wurde
 - -1, wenn das Spiel vorbei ist und verloren wurde
 - 0, wenn das Spiel noch nicht vorbei ist.—>Maximiere Summe der Beiträge



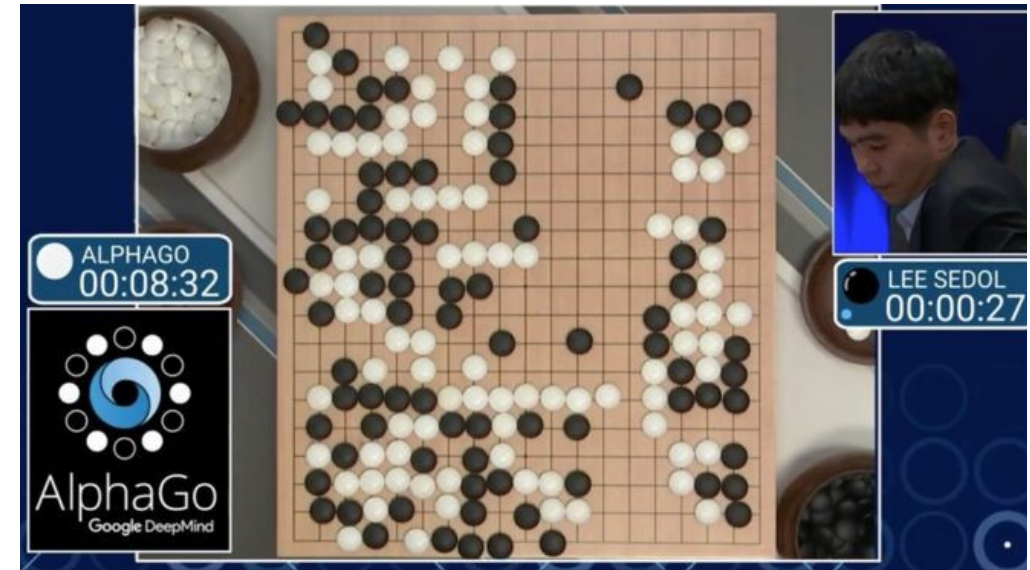
Policy Brettspiele



Wenn wir eine **Funktion** $Q(x_t, u_t)$ kennen würden, die jeden möglichen Zustand **bewertet** („wie viel Beitrag zur Zielfunktion kann man ausgehend von diesem Zustand insgesamt noch erwarten?“), wäre die Policy einfach:

Triff die Entscheidung mit der besten Bewertung:

$$\pi(x_t) = \operatorname{argmax}_{u_t} \underbrace{Q_t(x_t, u_t)}_{\substack{\text{Durchschnittlicher Wert} \\ \text{zum Zeitpunkt } t \\ \text{in Zustand } x_t \text{ zu sein und} \\ \text{Entscheidung } u_t \text{ zu treffen}}}$$



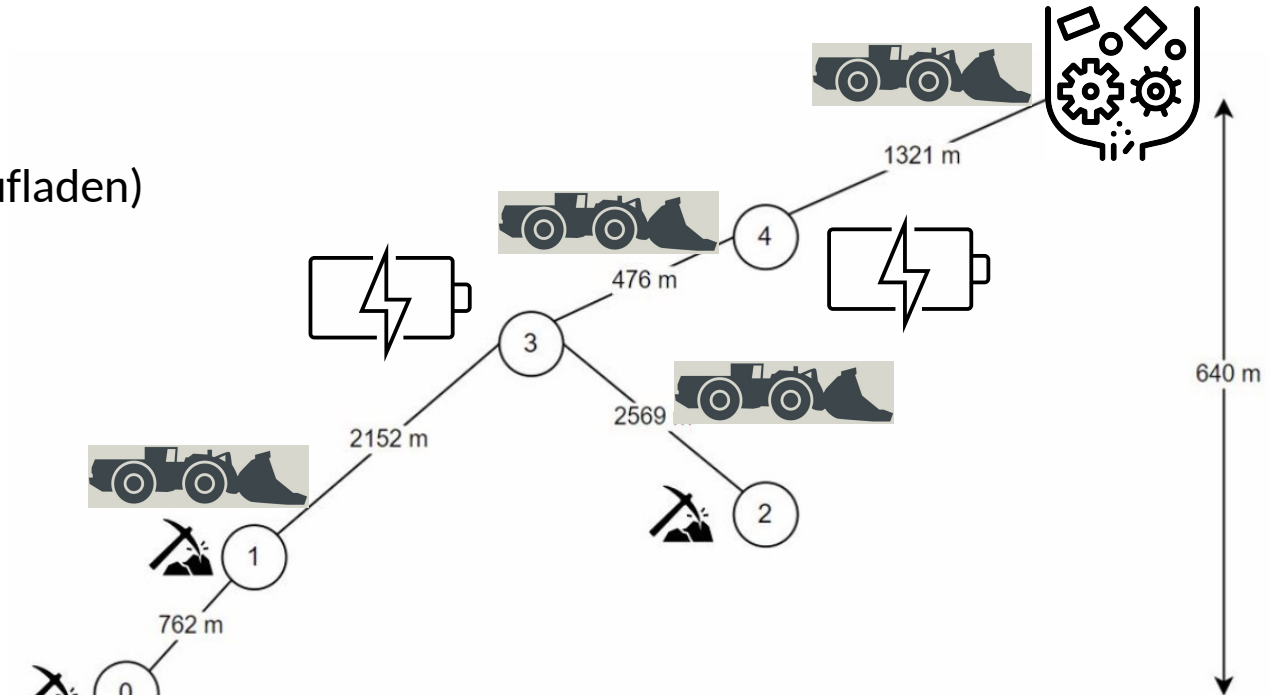
Lernen der Funktion Q : Grundlage von **Reinforcement Learning** („Trial and error++“)

DeepMind AlphaGo: Durch aufwendiges und geschicktes Sampling wird vorab die Q -Funktion gut approximiert. System kann jeden menschlichen Spieler besiegen.

Beispiel: Opportunity Charging



1. *Zustandsvariablen x_t :*
 - Für jeden Truck: Standort, Batteriestand (SoC), Gewicht
2. *Entscheidungsvariablen u_t ,*
 - Welche Aktion soll ausgeführt werden?
(Material laden/entladen, weiterfahren, Batterie aufladen)
3. *Exogene Information W_{t+1} :*
 - Veränderung Standort / Batteriestand
4. *Zielfunktion:*
 - Maximiere Menge bewegtes Material, Abzüge für Wartezeiten an Ladestationen und SoC nahe Null.
5. *Zustandsübergangsfunktion:*
 - U.a. einfaches physikalisches Modell für SoC



Variante 1: Parametrische Policy (Heuristik)

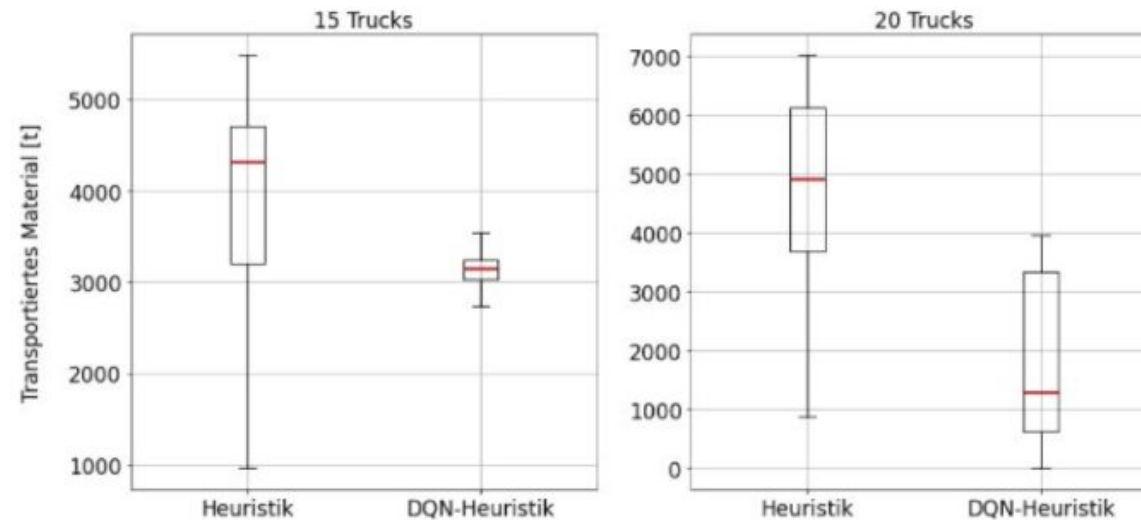
- + Wenn $\text{SoC} < w^{\text{laden}}$, fahre zur nächsten Ladesäule, wenn $\text{SoC} \geq w^{\text{laden}}$, hole Material.
- + Wenn $\text{SoC} < w^{\text{laden}} + \mu$, fahre zur nächsten Ladesäule, wenn sie frei ist.

Einfache Optimierung der Parameter w^{laden} und μ .

Variante 2: Reinforcement Learning (Deep Q-Learning)

Trainiere tiefes neuronales Netz, das den Wert einer Zustand-Entscheidungskombination vorhersagt, wähle die Entscheidung mit dem höchstem Q-Wert.

Aufwendiges Lernen der Q-Funktion.



Einfache Heuristik liefert durchweg bessere Resultate!

Policies: Zusammenfassung



Lösungen sequenzieller Entscheidungsprobleme sind **Policies**, mit denen man Entscheidungen trifft

Es gibt sehr unterschiedliche Arten von Policies:

1. Parametrisierte Funktion des Zustandes gibt Entscheidung vor.
2. Löse für jede Entscheidung ein deterministisches Optimierungsproblem.
3. Konstruiere vorab Approximation der Valuefunktion (sehr aufwendig).
Policy: Wähle den Zustand mit dem höchsten Wert.

Je nach Struktur des Problems können manche Arten sehr gut oder sehr schlecht geeignet sein.



Policy Typ A



Problem



Policy Typ B

Sequenzielle Entscheidungsprobleme vereinen u.a.

- + Statistik
- + Mathematische Modellierung
- + Optimierung
- + Machine Learning
- + „KI“

Historisch von unterschiedlichen Communities betrieben (RL, (stochastische) optimale Steuerung, Stochastische Programmierung, ...), Konsolidierung des Feldes in den letzten Jahren

Keine one-size-fits-all-Methode, sondern Sammlung von unterschiedlichen Ansätzen für das gleiche Problem